In [6]:
```
import os
import pandas as pd
from google.colab import drive
import matplotlib.pyplot as plt
import seaborn as sns
drive.mount('/content/drive')
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call dri
ve.mount("/content/drive", force_remount=True).
```

Helper Functions

In [7]:
```
# get mirex score for key
def get_mirex_score(target_key, target_mode, deteced_key, detected_mode):
    note_map = {'C': 0, 'C#': 1, 'Db': 1, 'D': 2, 'D#': 3, 'Eb': 3, 'E': 4, 'F':
                'F#': 6, 'Gb': 6, 'G': 7, 'G#': 8, 'Ab': 8, 'A': 9, 'A#': 10, 'B

    t_val, p_val = note_map.get(target_key), note_map.get(deteced_key)
    if t_val is None or p_val is None: return 0.0

    dist = (p_val - t_val) % 12

    if dist == 0 and target_mode == detected_mode: return 1.0 # Same
    if dist == 7 and target_mode == detected_mode: return 0.5 # 5th
    if target_mode == 'major' and detected_mode == 'minor' and dist == 9: return
    if target_mode == 'minor' and detected_mode == 'major' and dist == 3: return
    if dist == 0 and target_mode != detected_mode: return 0.2 # Parallel
    return 0.0

def mirex_key(target_key, target_mode, detected_key, detected_scale):

    score = get_mirex_score(target_key, target_mode, detected_key, detected_scal
    return score
```

In [8]:
```
def get_audio_file_features_data(filepath, model_name, audio_model_files):
    if model_name not in audio_model_files:
        audio_model_files[model_name] = {}

    feature_type = '_'.join(file.split('_')[1:-1]) # file convention is {model_n
    df = pd.read_csv(filepath)
    df['file_id'] = df['file_id'].str.split('_', n=1).str[1].replace('.wav', '',
    audio_model_files[model_name][feature_type] = df
    audio_model_files[model_name]['dataframe'] = None

    return audio_model_files
```

Load features extracted from audio files

In [9]:
```
# audio models
features_path = '/content/drive/MyDrive/fyp_audio/features'

audio_model_files = {}

for root, dir, files in os.walk(features_path):
    for file in files:
        filepath = os.path.join(root, file)
```

```
        modelname = file.split('_')[0]
        audio_model_files = get_audio_file_features_data(filepath, modelname, au
```

Ground truth loading

This loads the ground truth extraectred features and target values

In [10]:
```
df_features = pd.read_csv('/content/ground-truth-fma_features.csv')
df_bpm = pd.read_csv('/content/ground_truth_fma_bpm.csv')
df_key_mode = pd.read_csv('/content/ground_truth_fma_key-mode.csv')

ground_truth_bpm_df = pd.merge(
    df_features,
    df_bpm,
    left_on='file_id',
    right_on='track_id'
)

ground_truth_key_mode_df = pd.merge(
    df_features,
    df_key_mode,
    left_on='file_id',
    right_on='track_id'
)
```

In [11]:
```
# need to refactor to make this calculation of features modular across ground t
# 1. Initialize the dictionary structure to match your models
ground_truth_results = {
    'processed_tasks': {}
}

# --- TEMPO PROCESSING ---
tempo_task_df = ground_truth_bpm_df.copy()

# Rename columns to match your model script's expectation (adding leading unders
tempo_task_df = tempo_task_df.rename(columns={
    'bpm_essentia': '_bpm_essentia',
    'bpm_librosa': '_bpm_librosa',
    'bpm': 'target' # Ensure the target column is named 'target'
})

bpm_cols = [c for c in tempo_task_df.columns if '_bpm' in c]

for col in bpm_cols:
    measured = pd.to_numeric(tempo_task_df[col], errors='coerce')
    target = pd.to_numeric(tempo_task_df['target'], errors='coerce')

    tempo_task_df[f'{col}_abs_error'] = (measured - target).abs()
    tempo_task_df[f'{col}_signed_error'] = measured - target
    tempo_task_df[f'{col}_tempo_bias'] = tempo_task_df[f'{col}_signed_error'].ap
        lambda x: 'overshoot' if x > 0.5 else ('undershoot' if x < -0.5 else 'pe
    )

ground_truth_results['processed_tasks']['tempo'] = tempo_task_df
```

```
# --- KEY PROCESSING ---
key_task_df = ground_truth_key_mode_df.copy()

# Rename to match expected underscore format and standard 'target' name
key_task_df = key_task_df.rename(columns={
    'key': '_key',
    'scale': '_scale',
    'key_and_mode': 'target'
})

# Split target into key and mode
split_target = key_task_df['target'].str.split(' ', n=1, expand=True)
key_task_df['tmp_t_key'] = split_target[0]
key_task_df['tmp_t_mode'] = split_target[1]

# Apply Mirex Score (assuming mirex_key function is defined in your environment)
key_task_df['mirex_score'] = key_task_df.apply(
    lambda x: mirex_key(
        x['tmp_t_key'],
        x['tmp_t_mode'],
        x['_key'],
        x['_scale']
    ), axis=1
)

ground_truth_results['processed_tasks']['key'] = key_task_df

# 2. Add it to your main storage object
```

Model AUdio Loading

This loads the extracted features and target values into a DF for each model

Load master prompts to get the target values and join with target

In [13]:
```
master_prompts = '/content/master_prompts_v2.csv'
target_df = pd.read_csv(master_prompts)
target_df = target_df.rename(columns={"filename": "file_id"})

for model_name in audio_model_files:
    current_model_df = target_df.copy()
    for feature_type in audio_model_files[model_name]:
        if feature_type not in ['base_path', 'dataframe']:
            feature_df = audio_model_files[model_name][feature_type]
            prefixed_df = feature_df.add_prefix(f"{feature_type}_")
            prefixed_df = prefixed_df.rename(columns={f"{feature_type}_file_id":
            current_model_df = current_model_df.merge(prefixed_df, how='left', o

    audio_model_files[model_name]['dataframe'] = current_model_df
```

Process tasks

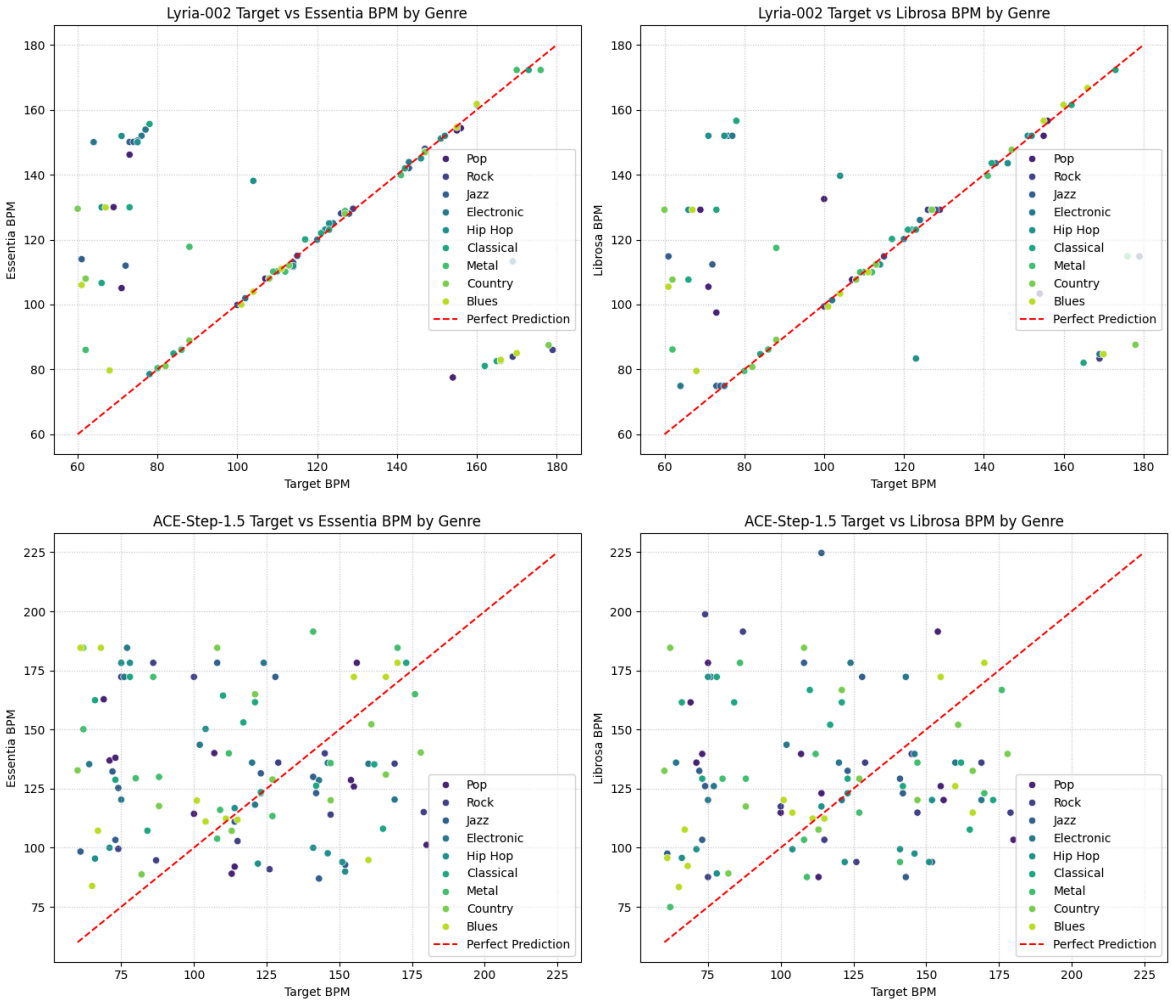
Here we are iterating over the tasks for each model, and extracting the features we need

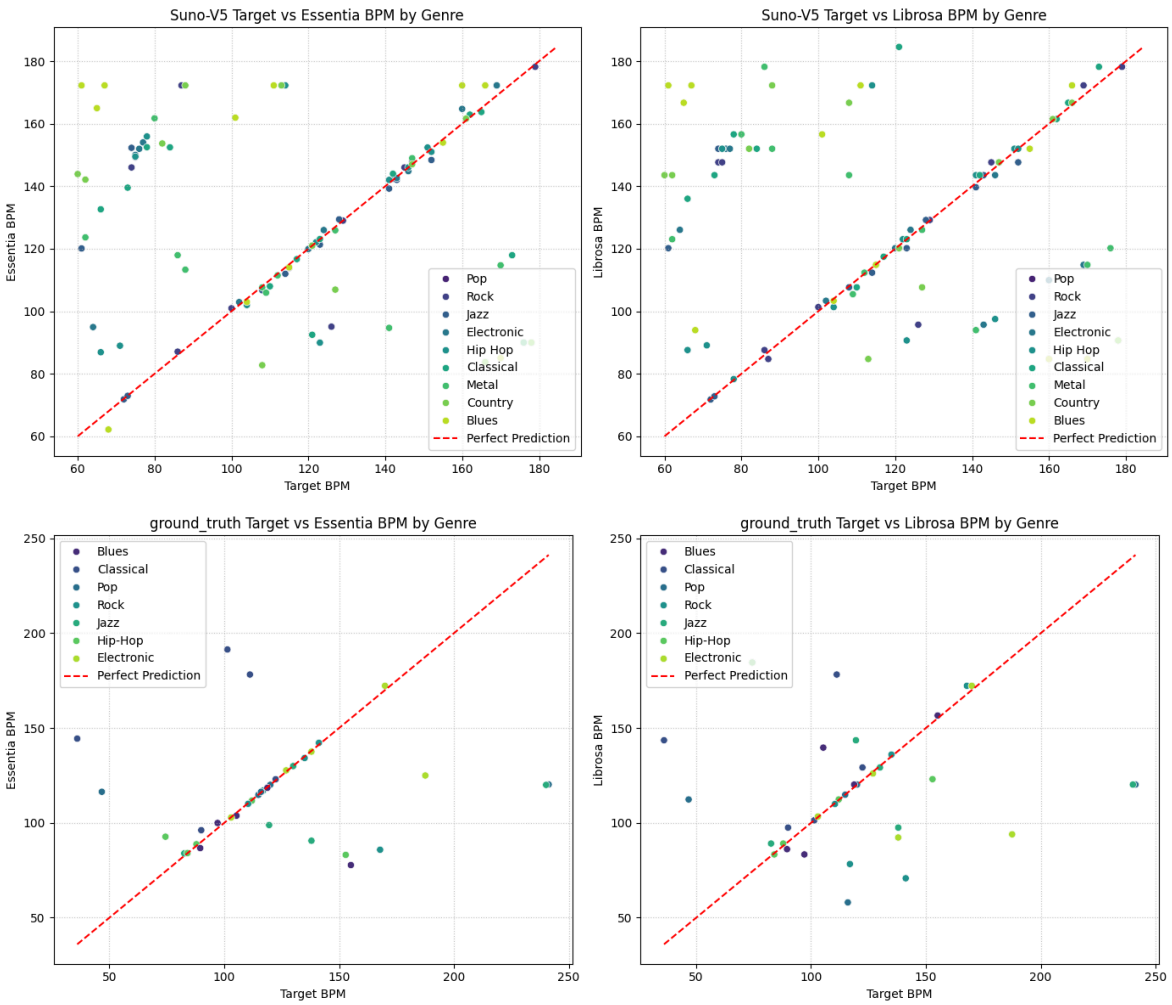
In [14]:
```
# the tasks are tempo, key, scale, tempo change, key modulation, time signature
import pandas as pd

for audio_model in audio_model_files:
    audio_model_df = audio_model_files[audio_model]['dataframe']
    processed_tasks = {}

    split_dfs = {task: group.copy() for task, group in audio_model_df.groupby('t

    for task, task_df in split_dfs.items():
        # calculates absolute and signed error for BPM
        if task == 'tempo':
            bpm_cols = [c for c in task_df.columns if '_bpm' in c]

            for col in bpm_cols:
                measured = pd.to_numeric(task_df[col], errors='coerce')
                target = pd.to_numeric(task_df['target'], errors='coerce')

                task_df[f'{col}_abs_error'] = (measured - target).abs()
                task_df[f'{col}_signed_error'] = measured - target

                task_df[f'{col}_tempo_bias'] = task_df[f'{col}_signed_error'].ap
                    lambda x: 'overshoot' if x > 0.5 else ('undershoot' if x < -
                )

        # calculates mirex
        if task == 'key':
            split_target = task_df['target'].str.split(' ', n=1, expand=True)
            task_df['tmp_t_key'] = split_target[0]
            task_df['tmp_t_mode'] = split_target[1]

            # 2. Use axis=1 to iterate row by row
            task_df['mirex_score'] = task_df.apply(
                lambda x: mirex_key(
                    x['tmp_t_key'],
                    x['tmp_t_mode'],
                    x['_key'],
                    x['_scale']
                ), axis=1
            )

        processed_tasks[task] = task_df

    audio_model_files[audio_model]['processed_tasks'] = processed_tasks

audio_model_files['ground_truth'] = ground_truth_results
```

Scatter Plots for Target vs Actual BPM

In [15]:
```
for model_name, model_data in audio_model_files.items():
    tempo_df = model_data['processed_tasks']['tempo'].copy()
    tempo_df['target'] = pd.to_numeric(tempo_df['target'], errors='coerce')
    print()
    plt.figure(figsize=(14, 6))

    # Diagonal line
    min_bpm = min(tempo_df['target'].min(), tempo_df['_bpm_essentia'].min(), tem
```

```
    max_bpm = max(tempo_df['target'].max(), tempo_df['_bpm_essentia'].max(), tem
    diagonal_line = [min_bpm, max_bpm]

    # Essentia BPM plotting
    plt.subplot(1, 2, 1)
    sns.scatterplot(data=tempo_df, x='target', y='_bpm_essentia', hue='genre', p
    plt.plot(diagonal_line, diagonal_line, color='red', linestyle='--', label='P
    plt.title(f'{model_name} Target vs Essentia BPM by Genre')
    plt.xlabel('Target BPM')
    plt.ylabel('Essentia BPM')
    plt.legend()
    plt.grid(True, linestyle=':', alpha=0.7)

    # Librosa BPM plotting
    plt.subplot(1, 2, 2)
    sns.scatterplot(data=tempo_df, x='target', y='_bpm_librosa', hue='genre', pa
    plt.plot(diagonal_line, diagonal_line, color='red', linestyle='--', label='P
    plt.title(f'{model_name} Target vs Librosa BPM by Genre')
    plt.xlabel('Target BPM')
    plt.ylabel('Librosa BPM')
    plt.legend()
    plt.grid(True, linestyle=':', alpha=0.7)

    plt.tight_layout()
    plt.show()
```



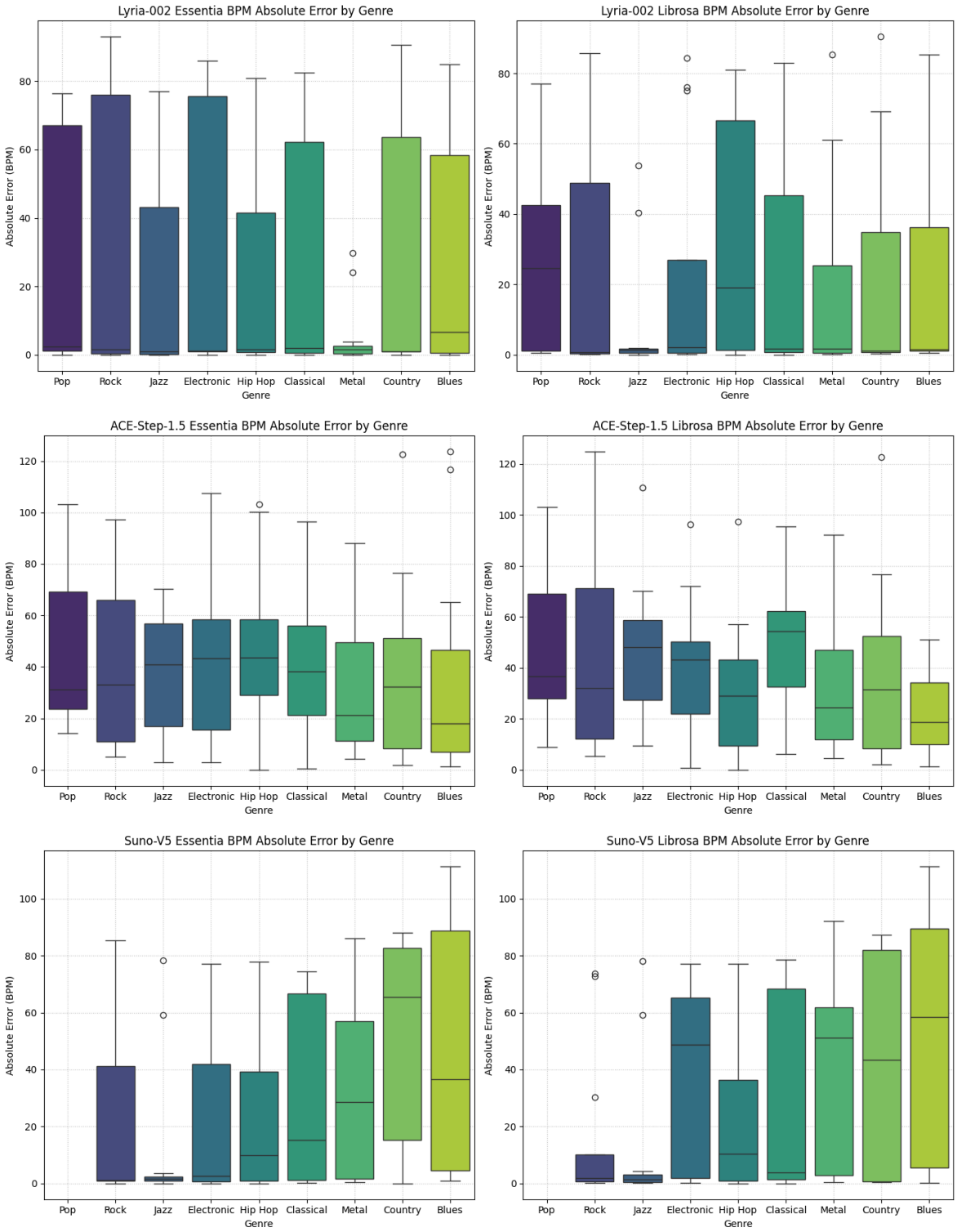


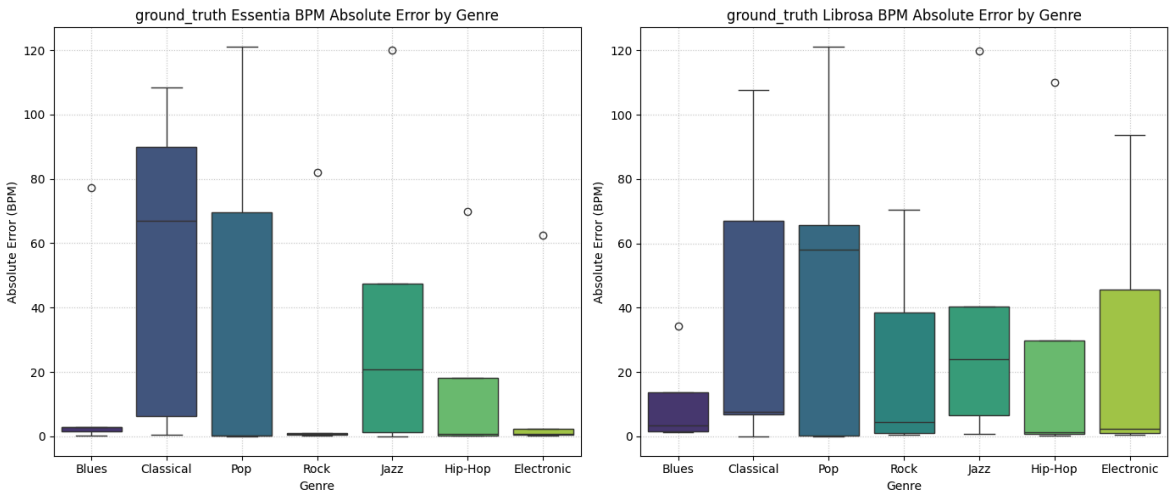
Box Plots for Absolute BPM Error by Genre

This shows a box plot for the difference between the target BPM and the measured BPM

In [16]:
```
for model_name, model_data in audio_model_files.items():
    tempo_df = model_data['processed_tasks']['tempo'].copy()
    tempo_df['target'] = pd.to_numeric(tempo_df['target'], errors='coerce')

    plt.figure(figsize=(14, 6))

    # Box plot for Essentia BPM absolute errors by genre
    plt.subplot(1, 2, 1)
    sns.boxplot(data=tempo_df, x='genre', y='_bpm_essentia_abs_error', hue='genr
    plt.title(f'{model_name} Essentia BPM Absolute Error by Genre')
    plt.xlabel('Genre')
    plt.ylabel('Absolute Error (BPM)')
    plt.grid(True, linestyle=':', alpha=0.7)

    # Box plot for Librosa BPM absolute errors by genre
    plt.subplot(1, 2, 2)
    sns.boxplot(data=tempo_df, x='genre', y='_bpm_librosa_abs_error', hue='genre
    plt.title(f'{model_name} Librosa BPM Absolute Error by Genre')
    plt.xlabel('Genre')
    plt.ylabel('Absolute Error (BPM)')
    plt.grid(True, linestyle=':', alpha=0.7)
```

```
plt.tight_layout()
plt.show()
```

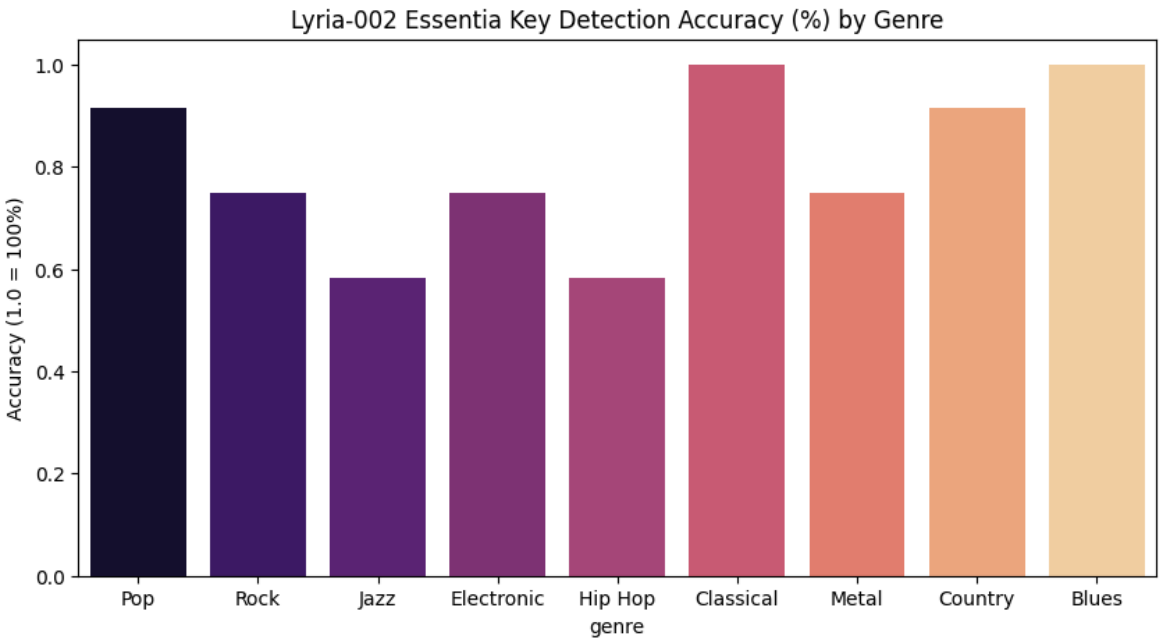




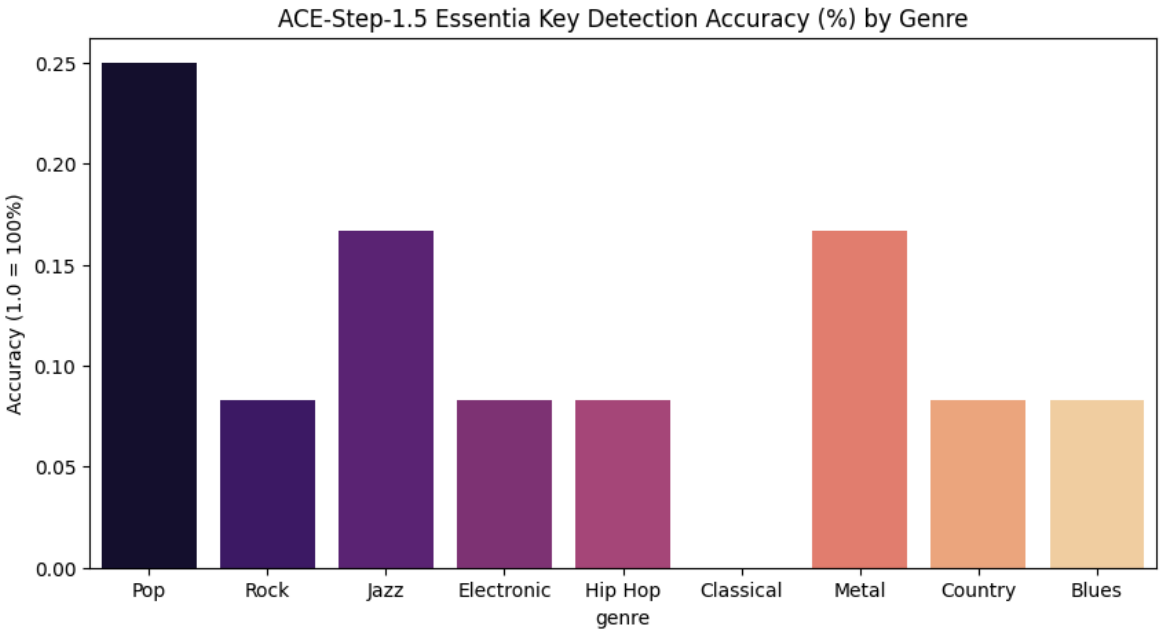
Bar Chart for key

In [17]:
```
for model_name, model_data in audio_model_files.items():
    key_df = model_data['processed_tasks']['key'].copy()
    key_df['target_root'] = key_df['target'].str.split(' ').str[0]
    key_df['is_correct'] = (key_df['target_root'].str.strip() == key_df['_key'].

    plt.figure(figsize=(10, 5))
    sns.barplot(data=key_df, x='genre', y='is_correct', palette='magma', errorba

    plt.title(f'{model_name} Essentia Key Detection Accuracy (%) by Genre')
    plt.ylabel('Accuracy (1.0 = 100%)')
    # plt.axhline(0.5, color='red', linestyle='--', alpha=0.5) # Benchmark line
    plt.show()
```

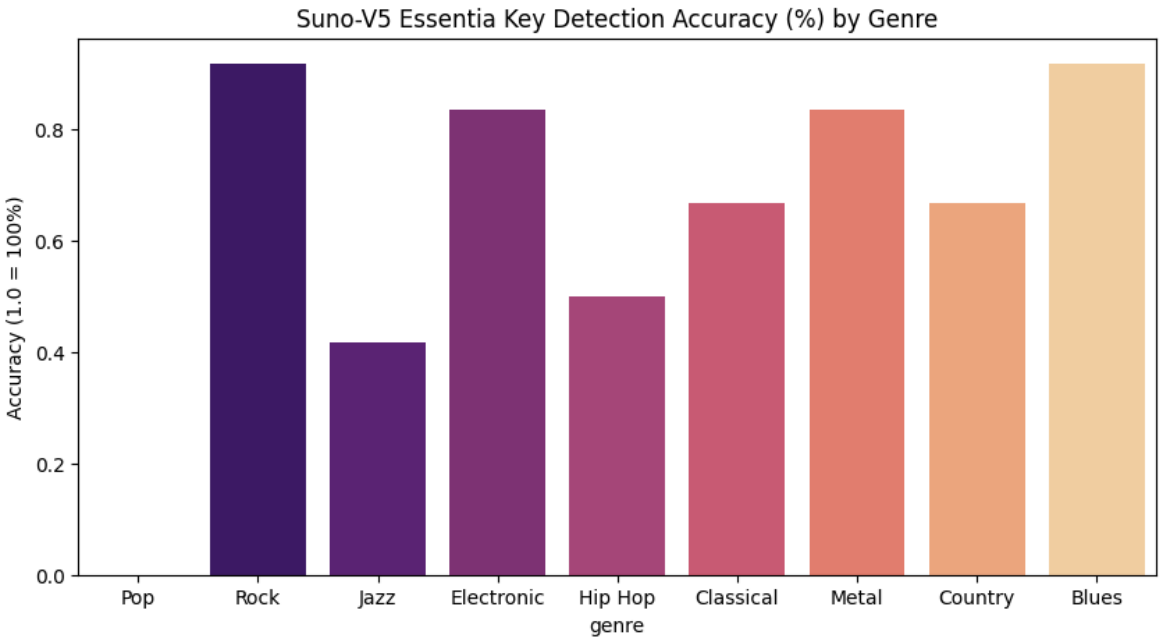
```
/tmp/ipykernel_16455/2917354457.py:7: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v
0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effe
ct.

  sns.barplot(data=key_df, x='genre', y='is_correct', palette='magma', errorbar=N
one)
```



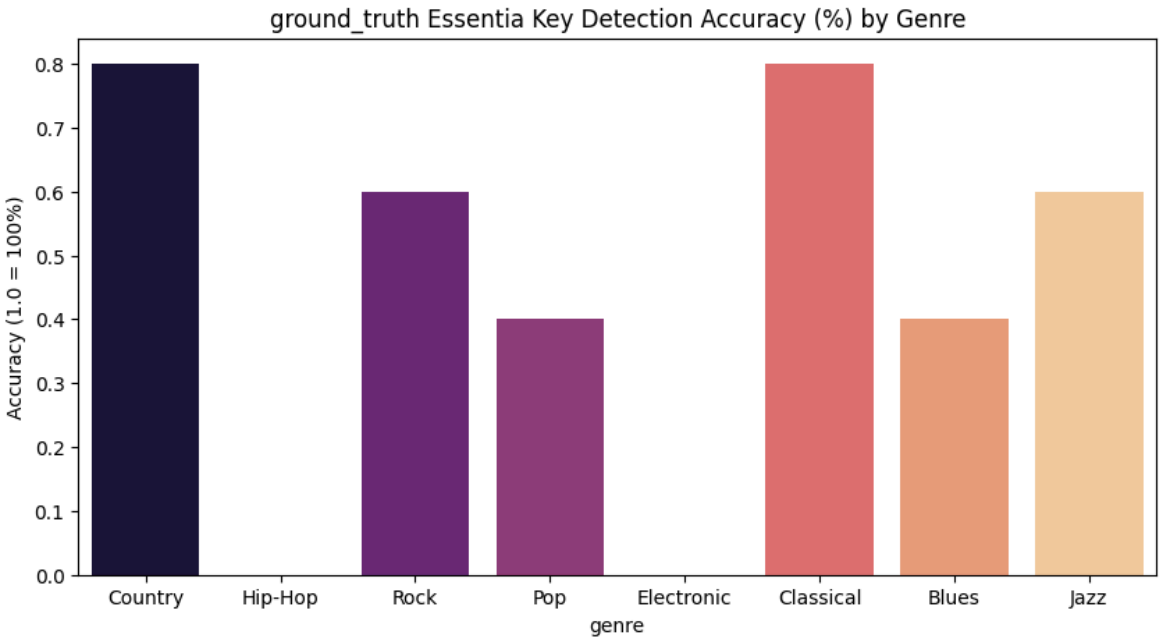
```
/tmp/ipykernel_16455/2917354457.py:7: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v
0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effe
ct.

  sns.barplot(data=key_df, x='genre', y='is_correct', palette='magma', errorbar=N
one)
```




```
/tmp/ipykernel_16455/2917354457.py:7: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v
0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effe
ct.

  sns.barplot(data=key_df, x='genre', y='is_correct', palette='magma', errorbar=N
one)
```



```
/tmp/ipykernel_16455/2917354457.py:7: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v
0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effe
ct.

  sns.barplot(data=key_df, x='genre', y='is_correct', palette='magma', errorbar=N
one)
```



Mirex Key Scores

In [18]:
```
import os
import csv

for model_name, model_data in audio_model_files.items():
    key_df = model_data['proccessed_tasks']['key'].copy()

    key_df['target_root'] = key_df['target'].str.split(' ').str[0]
    key_df['target_score'] = key_df['mirex_score']

    plt.figure(figsize=(10, 5))
    sns.barplot(data=key_df, x='genre', y='is_correct', palette='magma', errorba

    plt.title(f'{model_name} Essentia Key Detection Accuracy (%) by Genre')
    plt.ylabel('Accuracy (1.0 = 100%)')
    # plt.axhline(0.5, color='red', linestyle='--', alpha=0.5) # Benchmark line
    plt.show()
```

```
---------------------------------------------------------------------------
KeyError                                  Traceback (most recent call last)
/tmp/ipykernel_16455/3209204854.py in <cell line: 0>()
      3
      4 for model_name, model_data in audio_model_files.items():
----> 5     key_df = model_data['proccessed_tasks']['key'].copy()
      6
      7     key_df['target_root'] = key_df['target'].str.split(' ').str[0]

KeyError: 'proccessed_tasks'
```